

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЁТ ПО ЛАБОРАТОРНЫМ РАБОТАМ № 1-5
по дисциплине Теория графов

«Алгоритмы на графах»
Вариант 1

Студент,
группы 5130201/40003

_____ Адиатуллин Т.Р

Доцент

_____ Востров А.В

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Математическое описание	6
1.1 Граф и его определение	6
1.2 Распределение классического Вейбулла	6
1.3 Генерация связного ациклического графа по степеням вершин	6
1.4 Эксцентриситет, центр и диаметральные вершины	7
1.5 Метод Шимбелла	7
1.6 Упорядочивание графа по алгоритму Фалкерсона	8
1.7 Нахождение кратчайшего пути. Алгоритм Флойда-Уоршалла	8
1.8 Определение сети	9
1.9 Поток в сети	9
1.10 Алгоритм Форда–Фалкерсона	9
1.11 Нахождение заданного потока минимальной стоимости	10
1.12 Теорема Кирхгофа	10
1.13 Алгоритм Краскала	11
1.14 Код Прюфера	11
1.15 Максимальное независимое множество вершин	11
2 Особенности реализации	12
2.1 Общий поток выполнения	12
2.2 Модуль common	12
2.2.1 Graph<T>: шаблонный граф	12
2.2.2 WeibullDistribution: генерация случайных чисел	13
2.2.3 Generator<T>: генерация графов	13
2.3 Модуль lab1	14
2.3.1 Эксцентриситеты, центр и диаметральные вершины	14
2.3.2 Shimbelsolver<T>: метод Шимбелла	15
2.3.3 Поиск всех маршрутов (DFS с возвратом)	16
2.4 Модуль lab2	17
2.4.1 Fulkerson<T>: алгоритм Фалкерсона	17
2.4.2 FloydWarshall<T>: алгоритм Флойда-Уоршалла	18
2.4.3 Сравнение алгоритмов по числу итераций	19
2.5 Модуль lab3	19
2.5.1 MaxFlow<T>: алгоритм Форда–Фалкерсона (Эдмондс–Карп)	19
2.5.2 MinCostFlow<T>: поток минимальной стоимости	20
2.5.3 Генерация матриц пропускных способностей и стоимости	21
2.6 Модуль lab4	21
2.6.1 Kirchhoff: число остовных деревьев	21
2.6.2 Kruskal: алгоритм Краскала	22
2.6.3 Prufer: код Прюфера	22
2.6.4 MIS: максимальное независимое множество	23

3	Результаты работы программы	25
3.1	Общее меню и генерация графа	25
3.2	Лабораторная работа 1	25
3.3	Лабораторная работа 2	25
3.4	Лабораторная работа 3	25
3.5	Лабораторная работа 4	25
	Заключение	29
	Список литературы	32

Введение

Задание лабораторной работы 1.

1. Сформировать случайным образом связный ациклический ориентированный граф в соответствии с заданным распределением (параметры распределения задаются как константы и подбираются экспериментально) с введенным пользователем количеством вершин. Распределение брать из справочника Вадзинского: реализовать генерирующую формулу распределения Вейбулла. Генерация происходит по степеням вершин.
2. Посчитать эксцентриситеты вершин, выделить центр графа и диаметральные вершины.
3. Реализовать метод Шимбелла для сгенерированной с помощью заданного распределения весовой матрицы (пользователь вводит количество рёбер), в ответе представить минимальный и/или максимальный путь (в виде матрицы). При генерации необходимо учесть генерацию только положительных значений, только отрицательных значений и смешанную (на выбор пользователя).
4. Определить возможность построения маршрута от одной заданной точки до другой (вершины вводит пользователь) и указывать количество таковых маршрутов.
5. Реализовать меню для управления всеми пунктами.

Задание лабораторной работы 2.

1. Для заданных графов (случайно сгенерированных в предыдущей работе) реализовать упорядочивание графа по алгоритму Фалкерсона.
2. Сгенерировать весовую матрицу (положительную или с отрицательными весами — выбирает пользователь) и найти кратчайший путь для двух выбранных пользователем вершин. В результате выводится не только расстояние, но и сам путь в виде последовательности вершин. Реализовать алгоритм Флойда-Уоршалла с выводом матрицы расстояний.
3. Сравнить скорости работы реализованных алгоритмов (по количеству итераций).

Задание лабораторной работы 3.

1. На основе сгенерированного ориентированного графа получить случайным образом матрицы пропускных способностей и стоимости.
2. Для полученного графа найти максимальный поток по алгоритму Форда–Фалкерсона.

3. Вычислить заданный поток минимальной стоимости (в качестве величины потока брать значение, равное $\lfloor 2/3 \cdot \text{max} \rfloor$, где max — максимальный поток). Для поиска кратчайшего пути по стоимости использовать алгоритм Флойда-Уоршалла.

Задание лабораторной работы 4.

1. Для заданных неориентированных графов (случайно сгенерированных в первой работе) найти число остовных деревьев, используя матричную теорему Кирхгофа.
2. Построить минимальный по весу остов для сгенерированного (неориентированного) взвешенного графа, используя алгоритм Краскала. Полученный остов закодировать с помощью кода Прюфера и декодировать его; веса сохраняются при кодировании.
3. Реализовать алгоритм нахождения максимального независимого множества вершин на исходном графе и полученном остове (по выбору пользователя).

Реализация выполнена на C++17. Проект разбит на шаблонные библиотеки, собираемые через CMake. Единая точка входа — консольное меню, объединяющее все четыре лабораторных работы.

1. Математическое описание

1.1. Граф и его определение

Граф — пара $G = (V, E)$, где V — конечное непустое множество вершин, $E \subseteq V \times V$ — множество рёбер. Граф называется ориентированным, если каждое ребро — упорядоченная пара, и неориентированным в противном случае. Ациклический граф не содержит циклов; связный ациклический неориентированный граф является деревом, ориентированный — DAG (directed acyclic graph).

Взвешенный граф задаёт функцию весов $w : E \rightarrow \mathbb{R}$, ставящую каждому ребру вещественное число. В данной работе веса генерируются по распределению Вейбулла.

1.2. Распределение классического Вейбулла

Согласно справочнику Вадзинского, распределение Вейбулла задаётся плотностью вероятности:

$$f(x) = \frac{c}{a} \left(\frac{x}{a}\right)^{c-1} \exp\left(-\left(\frac{x}{a}\right)^c\right), \quad x > 0,$$

где $a > 0$ — параметр масштаба (*характерное время жизни*), $c > 0$ — параметр формы. При любом $c > 0$ $P\{X < a\} = 1 - e^{-1} \approx 0.6321$.

Для генерации случайных значений используется метод обратного преобразования. Сначала генерируется равномерное число $U \sim \text{Uniform}(0, 1)$, после чего применяется обратная функция распределения Вейбулла:

$$X = a(-\ln(1 - U))^{1/c}.$$

Это позволяет получить значения, распределённые по закону Вейбулла, что делает генерацию параметров графа более реалистичной и менее равномерной по сравнению с обычным генератором случайных чисел.

В реализации используются два независимых экземпляра распределения с параметрами $a = 10$, $c = 2$: один для генерации структуры графа (степеней вершин), другой — для генерации весов рёбер.

1.3. Генерация связного ациклического графа по степеням вершин

Пользователь задаёт число вершин n , тип ориентированности и знак весов. Число рёбер не вводится напрямую.

Шаг 1: генерация целевых степеней. Для каждой вершины $i \in \{0, \dots, n-1\}$ независимо сэмпляется значение по распределению Вейбулла, округляемое до целого и зажатое в диапазон $[1, \lfloor \log n \rfloor]$:

$$d_i = \max(1, \min(\lfloor X_i \rfloor, \lfloor \log n \rfloor)), \quad X_i \sim \text{Weibull}(a, c).$$

Шаг 2: формирование остовного дерева. Задаётся перестановка вершин π (в данной реализации — тождественная, чтобы сохранялась совместимость с топологической сортировкой). Для каждой вершины $\pi[i]$ при $i \geq 1$ выбирается предок случайно из $\{\pi[0], \dots, \pi[i-1]\}$ и добавляется ребро. Построение по индексам $j < i$ гарантирует ациклическость: ни одно ребро не идёт «назад» в топологическом порядке.

Шаг 3: добирание дополнительных рёбер. Для каждой вершины $\pi[i]$ с оставшимся бюджетом степени $d_i > 0$ формируется множество допустимых целей $\{j \mid j > i, (\pi[i], \pi[j]) \notin E\}$. Это множество перемешивается с помощью алгоритма Фишера-Йетса, где индексы выбираются по распределению Вейбулла (*weibullShuffle*), после чего добавляются первые $\min(d_i, |\text{допустимых}|)$ рёбер.

Ориентированность при добавлении рёбер учитывается явно: для неориентированных графов каждое ребро добавляется симметрично.

1.4. Эксцентриситет, центр и диаметральные вершины

Эксцентриситетом $e(v)$ вершины v в связном графе $G(V, E)$ называется максимальное расстояние от вершины v до других вершин графа G :

$$e(v) \stackrel{\text{Def}}{=} \max_{u \in V} d(v, u).$$

Радиусом $R(G)$ графа G называется наименьший из эксцентриситетов вершин:

$$R(G) \stackrel{\text{Def}}{=} \min_{v \in V} e(v).$$

Вершина v называется *центральной*, если её эксцентриситет совпадает с радиусом графа, $e(v) = R(G)$. Множество центральных вершин называется *центром* графа:

$$C(G) \stackrel{\text{Def}}{=} \{v \in V \mid e(v) = R(G)\}.$$

Диаметральные вершины — с максимальным эксцентриситетом:

$$D(G) = \{v \mid e(v) = \text{diam}(G)\}.$$

Расстояния вычисляются алгоритмом обхода в ширину (*BFS*), запускаемым из каждой вершины; алгоритмическая сложность линейная ($O(V + E)$) для одного прохода, а итоговая сложность квадратична $O(V(V + E))$.

1.5. Метод Шимбелла

1. Операция «умножения» двух величин a и b при возведении матрицы в степень соответствует их алгебраической сумме:

$$a \cdot b = b \cdot a \Rightarrow a + b = b + a, \quad a \cdot 0 = 0 \cdot a = 0 \Rightarrow a + 0 = 0. \quad (1)$$

2. Операция «сложения» двух величин a и b заменяется выбором минимального (максимального) элемента из ненулевых:

$$a + b = b + a = \min(\max)\{a, b\}. \quad (2)$$

Нули при этом игнорируются: минимальный или максимальный элемент выбирается из ненулевых значений. Нуль в результате операции может быть получен лишь тогда, когда все слагаемые — нулевые.

Элемент матрицы $A^{(2)}$, полученной возведением A в степень 2, вычисляется как:

$$\omega_{ij}^{(2)} = \min(\max) \left\{ \omega_{i1}^{(1)} + \omega_{1j}^{(1)}, \omega_{i2}^{(1)} + \omega_{2j}^{(1)}, \dots, \omega_{in}^{(1)} + \omega_{nj}^{(1)} \right\}.$$

Матрица $A^{(k)}$, вычисленная последовательным умножением, содержит длины путей из ровно k рёбер для всех пар вершин. Временная сложность: $O(k \cdot n^3)$, пространственная: $O(n^2)$.

1.6. Упорядочивание графа по алгоритму Фалкерсона

Алгоритм Фалкерсона — графический метод топологической сортировки DAG, состоящий из следующих шагов.

1. Найти вершины графа, в которые не входит ни одна дуга. Они образуют первую группу. Пронумеровать вершины группы в произвольном порядке.
2. Вычеркнуть все пронумерованные вершины и дуги, из них исходящие. В получившемся графе найдётся, по крайней мере, одна вершина, в которую не входит ни одна дуга. Этой вершине, входящей во вторую группу, присвоить очередной номер и так далее. Второй шаг повторять до тех пор, пока не будут упорядочены все вершины.

Если на каком-либо шаге вершин без входящих дуг не оказалось, граф содержит цикл и алгоритм неприменим. Алгоритмическая сложность: $O(V^2)$ в матричном представлении, поскольку на каждом шаге выполняется просмотр всей матрицы смежности для поиска вершин без входящих дуг.

1.7. Нахождение кратчайшего пути. Алгоритм Флойда-Уоршалла

Алгоритм Флойда-Уоршалла находит кратчайшие пути между *всеми* парами вершин в ориентированном взвешенном графе.

Для хранения информации о путях используется матрица H размера $n \times n$:

$$H[i, j] = \begin{cases} k, & \text{если } k \text{ — первая вершина, достигаемая на кратчайшем пути из } i \text{ в } j; \\ -1, & \text{если из вершины } i \text{ в вершину } j \text{ нет пути.} \end{cases}$$

Основной цикл: для каждой промежуточной вершины k проверяется

если $d[i][k] + d[k][j] < d[i][j]$, то $d[i][j] \leftarrow d[i][k] + d[k][j]$, $H[i][j] \leftarrow H[i][k]$.

Если в G есть цикл с отрицательным весом, то решения поставленной задачи не существует.

Алгоритмическая сложность: $O(n^3)$, пространственная: $O(n^2)$.

1.8. Определение сети

Пусть $G = (V, E)$ — орграф с одним истоком $s \in V$ и одним стоком $t \in V$, где $s \neq t$. **Сетью** $S = (G; c)$ называется орграф G с заданной функцией $c : E \rightarrow \mathbb{R}_+$, сопоставляющей каждой дуге $e \in E$ неотрицательное действительное число $c(e)$ — пропускную способность.

1.9. Поток в сети

Потоком в сети S называется функция $f : E \rightarrow \mathbb{R}_+$, для которой выполняются два условия.

Первое — ограничение пропускной способности: для каждой дуги $e \in E$

$$f(e) \leq c(e).$$

Второе — закон сохранения потока: для каждой вершины $v \in V$, $v \neq s, t$,

$$D_f(v) = \sum_{e \in A(v)} f(e) - \sum_{e \in B(v)} f(e) = 0,$$

где $A(v)$ — множество дуг, исходящих из v , $B(v)$ — множество дуг, входящих в v .

Величиной потока f называется число

$$p_f = \sum_{e \in A(s)} f(e).$$

1.10. Алгоритм Форда–Фалкерсона

Теорема (Л. Р. Форд, Д. Р. Фалкерсон, 1962). *Величина максимального потока $p_{\max}$ в сети S совпадает с минимальной пропускной способностью $r_{\min}$ её разрезом.*

Алгоритм итеративно находит увеличивающий путь от истока к стоку в остаточной сети. Величина дополнительного потока по найденному пути:

$$\Delta f = \min_{(u,v) \in P} r(u, v).$$

Для каждого ребра (u, v) пути P поток обновляется:

$$f(u, v) \leftarrow f(u, v) + \Delta f, \quad f(v, u) \leftarrow f(v, u) - \Delta f.$$

Алгоритм завершается, когда увеличивающий путь не существует. В реализации поиск пути выполняется через BFS (алгоритм Эдмондса–Карпа), что гарантирует сложность $O(V \cdot E^2)$.

1.11. Нахождение заданного потока минимальной стоимости

Рассмотрим задачу определения потока заданной величины θ от s к t в сети $G = (S, U, \Omega, D)$, в которой каждая дуга (x_i, x_j) характеризуется пропускной способностью $c(x_i, x_j) \in \Omega$ и неотрицательной стоимостью $d(x_i, x_j) \in D$.

Математическая модель задачи: минимизировать

$$\sum_{(x_i, x_j) \in U} d(x_i, x_j) \cdot \varphi(x_i, x_j),$$

при ограничениях:

1. $0 \leq \varphi(x_i, x_j) \leq c(x_i, x_j)$ для всех $(x_i, x_j) \in U$;
2. для истока s : $\sum_{x_j} \varphi(s, x_j) - \sum_{x_j} \varphi(x_j, s) = \theta$
3. для стока t : $\sum_{x_j} \varphi(t, x_j) - \sum_{x_j} \varphi(x_j, t) = -\theta$
4. для любой другой вершины: условие сохранения потока равно нулю.

1.12. Теорема Кирхгофа

Пусть $G = (V, E)$ — связный неориентированный граф без петель и кратных рёбер, $|V| = n$. Матрица Кирхгофа $B = D - A(G)$, где $D = \text{diag}(\deg v_1, \dots, \deg v_n)$ — диагональная матрица степеней вершин, $A(G)$ — матрица смежности:

$$b_{ij} = \begin{cases} \deg v_i, & i = j, \\ -1, & i \neq j, v_i \sim v_j, \\ 0, & i \neq j, v_i \not\sim v_j. \end{cases}$$

Теорема (матричная теорема Кирхгофа). Число различных остовных деревьев связного графа G равно определителю любого минора матрицы Кирхгофа:

$$\tau(G) = \det B_{ii},$$

где B_{ii} — матрица B с удалённой i -й строкой и i -м столбцом.

В реализации определитель вычисляется алгоритмом Барейсса — точным целочисленным методом устранения Гаусса без ошибок округления. Алгоритмическая сложность: $O(n^3)$, пространственная: $O(n^2)$.

1.13. Алгоритм Краскала

Будем последовательно строить подграф F графа G («растущий лес»), пытаясь на каждом шаге достроить F до некоторого MST. Начнём с того, что включим в F все вершины графа G . Теперь будем обходить множество $E(G)$ в порядке неубывания весов рёбер. Если очередное ребро e соединяет вершины одной компоненты связности F , то добавление его в остов приведёт к возникновению цикла — e не включается. Иначе e соединяет разные компоненты связности, и из леммы о безопасном ребре следует, что e является безопасным — добавляем его в F .

Для проверки цикличности в реализации используется BFS по текущему остову. Алгоритмическая сложность: $O(E^2)$ (проверка через BFS), пространственная: $O(V + E)$.

1.14. Код Прюфера

Кодирование. Пусть $T = (V, E_T)$ — дерево с вершинами $V = \{0, \dots, n-1\}$. Код Прюфера — последовательность $A = (A[1], \dots, A[n-2])$: на каждом из $n-2$ шагов выбирается висячая вершина v с минимальным номером ($\deg v = 1$), в код записывается её единственный сосед $\Gamma(v)$, после чего вершина v удаляется из дерева. Вместе с кодом сохраняются веса рёбер.

Декодирование. По коду $A = (A[1], \dots, A[n-2])$ восстанавливается дерево $T = (V, E_T)$. Поддерживается множество «свободных» вершин B . На каждом шаге i выбирается наименьшая вершина $v \in B$, не встречающаяся в остатке кода $A[i..n-2]$; добавляется ребро $(v, A[i])$, вершина v удаляется из B . Веса восстанавливаются из сохранённого словаря. Сложность кодирования и декодирования: $O(n^2)$, пространственная: $O(n)$.

1.15. Максимальное независимое множество вершин

Независимым множеством графа $G = (V, E)$ называется подмножество $S \subseteq V$ такое, что никакие два его элемента не соединены ребром:

$$\forall u, v \in S : (u, v) \notin E.$$

Максимальным независимым множеством (MIS) называется такое независимое множество S^* , что $|S^*| \geq |S|$ для любого другого независимого множества S .

Нахождение MIS является NP-трудной задачей в общем случае. В данной работе реализован точный алгоритм на основе полного перебора с отсечениями (backtracking): рекурсивно перебираются кандидаты, каждый из которых добавляется в текущее множество S , если он не смежен ни с одной уже включённой вершиной. После добавления вершины v из кандидатов исключаются все её соседи. Сложность: $O(2^n)$ в худшем случае, пространственная: $O(n)$.

2. Особенности реализации

2.1. Общий поток выполнения

Проект собирается через CMake. Точка входа — единый исполняемый файл, который запускает консольное меню. Зависимости между модулями однонаправленные: лабораторные зависят от `common`, `common` не знает ни о каких лабораторных.

Типичный сценарий: пользователь выбирает пункт меню, `main.cpp` вызывает соответствующий метод `LabXUI`, который читает параметры, делегирует работу алгоритмическому классу и выводит результат в консоль.

```
main.cpp -> LabXUI -> Graph<T> / Generator<T> ->
        -> Algorithms (Shimbel, Fulkerson, Floyd, ...) -
> stdout
```

Рис. 1: Поток выполнения

2.2. Модуль `common`

Модуль содержит весь код, переиспользуемый несколькими лабораторными: граф, генератор, распределение и утилиты.

2.2.1. `Graph<T>`: шаблонный граф

`Graph<T>` — шаблонный класс, параметризованный типом веса рёбер.

```
1 template <typename T>
2 class Graph {
3     private:
4         int numVertices;
5         vector<vector<int>> adjMatrix;
6         vector<vector<T>> weightMatrix;
7     public:
8         Graph(int n);
9         void addEdge(int from, int to, T weight);
10        bool hasEdge(int from, int to) const;
11        T getEdge(int from, int to) const;
12        int getNumVertices() const;
13        vector<vector<int>> getAdjMatrix() const;
14        vector<vector<T>> getWeightMatrix() const;
15        Graph<T> reorder(const vector<int>& order) const;
16        // ...
17 };
```

Listing 1: Объявление `Graph` (`graph.h`)

Хранение данных. Граф хранится одновременно в двух матрицах размера $n \times n$. `adjMatrix[i][j] = 1` означает наличие ребра (i, j) ; `weightMatrix[i][j]` хранит соответствующий вес. Такое представление даёт $O(1)$ проверку наличия ребра и $O(1)$ доступ к весу, однако занимает $O(n^2)$ памяти вне зависимости от плотности графа.

BFS. Метод `bfs(int start)` выполняет обход в ширину по матрице весов: ребро (v, to) считается существующим, если `weightMatrix[v][to] != T{}`. Результат — вектор расстояний (в рёбрах) от источника до всех вершин, ∞ для недостижимых.

DFS и поиск всех путей. Методы `dfs` и `findAllPaths` реализуют поиск с возвратом (backtracking). `dfs` рекурсивно обходит граф, поддерживая вектор посещённых вершин; при достижении цели текущий путь копируется в результат.

Метрики. Методы `eccentricity`, `allEccentricities`, `findCenter` и `findDiametral` вычисляют характеристики непосредственно внутри класса графа, используя BFS.

Перенумерация. Метод `reorder(order)` строит новый граф, переставляя вершины согласно переданной перестановке π — используется для отображения результата алгоритма Фалкерсона в виде упорядоченной матрицы смежности.

2.2.2. WeibullDistribution: генерация случайных чисел

Класс инкапсулирует распределение Вейбулла. Конструктор принимает параметры масштаба a и формы c ; по умолчанию $a = 1$, $c = 0.2$.

```
1 double WeibullDistribution::generate() {  
2     double u = distribution(generator);  
3     double raw = a * std::pow(-std::log(1 - u), 1.0 / c);  
4     return (raw > 0) ? raw : -raw;  
5 }
```

Listing 2: Генерация числа по распределению Вейбулла (weibull.cpp)

Метод генерирует равномерное $u \in (0, 1)$ через `std::mt19937` и применяет формулу обратного преобразования $X = a(-\ln(1 - u))^{1/c}$. Взятие модуля гарантирует положительность результата при любых параметрах.

2.2.3. Generator<T>: генерация графов

Шаблонный класс `Generator<T>` содержит два экземпляра `WeibullDistribution`: один для структуры графа (`distributionForStructure`), другой для ве-

сов рёбер (`distributionForWeights`). Параметры распределений передаются конструктору явно.

```

1 template <typename T>
2 T Generator<T>::generateWeight(WeightType weightType) {
3     double weight = distributionForWeights.generate();
4     if (weightType == WeightType::NEGATIVE)
5         return static_cast<T>(-weight);
6     else if (weightType == WeightType::MIXED)
7         return isNegative() ? static_cast<T>(-weight)
8                               : static_cast<T>(weight);
9     return static_cast<T>(weight);
10 }
11
12 template <typename T>
13 void Generator<T>::weibullShuffle(vector<int>& vertices) {
14     int n = vertices.size();
15     for (int i = n - 1; i > 0; i--) {
16         int j = randomIndex(i + 1);
17         swap(vertices[i], vertices[j]);
18     }
19 }

```

Listing 3: Генератор весов и Вейбуллово перемешивание (`generator.cpp`)

Метод `generateGraph()` строит граф в два прохода. В первом проходе формируется остоное дерево: для каждой вершины $i \geq 1$ предок выбирается случайно из $\{0, \dots, i - 1\}$, что гарантирует связность и ациклическость. Во втором проходе для каждой вершины с ненулевым бюджетом степени формируется множество допустимых целей (только $j > i$, ребро ещё не существует), множество перемешивается с помощью `weibullShuffle` и добавляются первые d_i рёбер.

2.3. Модуль `lab1`

2.3.1. Эксцентриситеты, центр и диаметральные вершины

Все метрики вычисляются внутри класса `Graph<T>`.

Вход: `Graph<T>` — связный граф.

Выход: вектор эксцентриситетов, списки центральных и диаметральных вершин.

Описание: Для каждой вершины запускается BFS (`bfs(int start)`) по матрице весов — ребро считается существующим, если вес ненулевой. Результат BFS — вектор расстояний `distances` (в рёбрах). Эксцентриситет вершины — максимум из достижимых расстояний:

```

1 template <typename T>
2 int Graph<T>::eccentricity(int vertex) const {
3     vector<int> dist = bfs(vertex);
4     const int INF = std::numeric_limits<int>::max();
5     int maxDist = 0;

```

```

6   for (int i = 0; i < numVertices; i++) {
7       if (i == vertex || dist[i] == INF) continue;
8       maxDist = std::max(maxDist, dist[i]);
9   }
10  return maxDist;
11 }

```

Listing 4: BFS и вычисление эксцентриситета (graph.cpp)

Центр определяется как множество вершин с минимальным эксцентриситетом (радиус графа), диаметральные — с максимальным (диаметр графа):

```

1  template <typename T>
2  vector<int> Graph<T>::findCenter() const {
3      vector<int> eccs = allEccentricities();
4      int radius = *std::min_element(eccs.begin(), eccs.end());
5      vector<int> centers;
6      for (int i = 0; i < numVertices; i++)
7          if (eccs[i] == radius) centers.push_back(i);
8      return centers;
9  }
10
11 template <typename T>
12 vector<int> Graph<T>::findDiametral() const {
13     vector<int> eccs = allEccentricities();
14     int diameter = 0;
15     for (int ecc : eccs)
16         if (ecc != std::numeric_limits<int>::max() && ecc >
17             diameter)
18             diameter = ecc;
19     vector<int> diametral;
20     for (int i = 0; i < numVertices; i++)
21         if (eccs[i] == diameter) diametral.push_back(i);
22     return diametral;
23 }

```

Listing 5: Поиск центра и диаметральных вершин (graph.cpp)

Сложность: $O(V(V + E))$ при матричном представлении, где обход BFS стоит $O(V^2)$ за один запуск.

2.3.2. ShimbelSolver<T>: метод Шимбелла

Вход: Graph<T> const& (конструктор), int k — длина пути в рёбрах.

Выход: матрица кратчайших путей и матрица наидлиннейших путей из ровно k рёбер для всех пар вершин.

Описание: Начальная матрица инициализируется по весовой матрице графа: $\text{currentMatrix}[i][j] = w(i, j)$ если ребро существует, иначе — сторожевое значение ($+\infty$ или $-\infty$). Матрица итеративно умножается $k - 1$ раз методом `multiplyMatrix(bool findMin)`. При умножении для каждой пары (i, j) перебираются все промежуточные вершины m : если $\text{currentMatrix}[i][m]$ не является сторожевым и $w[m][j] \neq 0$, вычисляется кандидат $= \text{currentMatrix}[i][m] +$

$w[m][j]$, после чего выбирается минимальный (при `findMin = true`) или максимальный кандидат:

```

1 template <typename T>
2 void Shimbelsolver<T>::multiplyMatrix(bool findMin) {
3     const T INF = getINF();
4     const T empty = findMin ? INF : -INF;
5     auto weight = graph.getWeightMatrix();
6     vector<vector<T>> next(n, vector<T>(n, empty));
7     for (int i = 0; i < n; i++)
8         for (int j = 0; j < n; j++)
9             for (int m = 0; m < n; m++) {
10                 if (currentMatrix[i][m] == empty
11                     || weight[m][j] == T{}) continue;
12                 T candidate = currentMatrix[i][m] + weight[m][j];
13                 if (findMin) next[i][j] = std::min(next[i][j],
14 candidate);
15                 else next[i][j] = std::max(next[i][j],
16 candidate);
17             }
18     currentMatrix = next;
19 }

```

Listing 6: Матричное умножение для метода Шимбелла (shimbel.cpp)

Краевой случай $k = 0$ возвращает единичную матрицу. Временная сложность: $O(k \cdot n^3)$, пространственная: $O(n^2)$.

2.3.3. Поиск всех маршрутов (DFS с возвратом)

Вход: `Graph<T> const&, int from, int to`.

Выход: `vector<vector<int>>` — все простые пути.

Описание: Метод `findAllPaths` инициализирует вектор посещённых вершин и запускает рекурсивный `dfs`. При достижении вершины `target` текущий путь `currentPath` копируется в `allPaths`. При выходе из рекурсии вершина снимается с отметки (*backtracking*):

```

1 template <typename T>
2 void Graph<T>::dfs(int current, int target,
3     vector<bool>& visited,
4     vector<int>& currentPath,
5     vector<vector<int>>& allPaths) const {
6     if (current == target) {
7         allPaths.push_back(currentPath);
8         return;
9     }
10    for (int next = 0; next < numVertices; next++) {
11        if (weightMatrix[current][next] != T{} && !visited[next]) {
12            visited[next] = true;
13            currentPath.push_back(next);
14            dfs(next, target, visited, currentPath, allPaths);
15            currentPath.pop_back();
16        }
17    }
18 }

```



```

16         visited[next] = false;
17     }
18 }
19 }

```

Listing 7: Поиск всех путей DFS (graph.cpp)

Сложность: $O(n!)$ в худшем случае, пространственная: $O(V)$.

2.4. Модуль lab2

2.4.1. Fulkerson<T>: алгоритм Фалкерсона

Вход: Graph<T> const& (конструктор).

Выход: vector<int> — вектор новых номеров вершин после топологической сортировки.

Описание: На каждой итерации вспомогательный метод findRoots просматривает матрицу смежности и собирает все ещё не вычеркнутые вершины без входящих дуг. Каждой такой вершине присваивается очередной номер, затем она помечается удалённой. Шаг повторяется до тех пор, пока не будут упорядочены все вершины. Счётчик итераций инкрементируется при каждом просмотре ячейки матрицы, что позволяет сравнивать алгоритмы по трудоёмкости.

Если на каком-либо шаге корни не найдены — граф содержит цикл и алгоритм неприменим; выводится соответствующее сообщение.

```

1  template <typename T>
2  vector<int> Fulkerson<T>::findRoots(
3      const vector<bool>& removed) const
4  {
5      auto adj = graph.getAdjMatrix();
6      vector<int> roots;
7      for (int v = 0; v < n; v++) {
8          if (removed[v]) continue;
9          bool hasIncoming = false;
10         for (int from = 0; from < n; from++) {
11             iterCount++;
12             if (!removed[from] && from != v
13                 && adj[from][v] != 0) {
14                 hasIncoming = true;
15                 break;
16             }
17         }
18         if (!hasIncoming) roots.push_back(v);
19     }
20     return roots;
21 }
22
23 template <typename T>
24 vector<int> Fulkerson<T>::computeOrder() const {
25     iterCount = 0;
26     vector<bool> removed(n, false);

```

```

27 vector<int> order(n, -1);
28 int counter = 0;
29 while (counter < n) {
30     vector<int> roots = findRoots(removed);
31     if (roots.empty()) { /* цикл в графе */ return {}; }
32     for (int v : roots) {
33         order[v] = counter++;
34         removed[v] = true;
35     }
36 }
37 return order;
38 }

```

Listing 8: Поиск корней и вычисление порядка (fulkerson.cpp)

По результату вызывается `Graph<T>::reorder(order)`, после чего выводятся перенумерованная матрица смежности и матрица весов. Алгоритмическая сложность: $O(V^2)$, пространственная: $O(V)$.

2.4.2. FloydWarshall<T>: алгоритм Флойда-Уоршалла

Вход: `Graph<T> const&` (конструктор).

Выход: матрица расстояний `distMatrix` и матрица следующих вершин `nextMatrix` для восстановления пути.

Описание: Инициализация: $d[i][i] = 0$, $d[i][j] = w(i, j)$ если ребро существует, иначе $d[i][j] = +\infty$; $next[i][j] = j$ при наличии ребра, -1 — иначе.

Основной тройной цикл перебирает промежуточные вершины k :

```

1 template <typename T>
2 void FloydWarshall<T>::compute() {
3     const T INF = getPosINF();
4     iterCount = 0;
5     for (int k = 0; k < n; k++)
6         for (int i = 0; i < n; i++)
7             for (int j = 0; j < n; j++) {
8                 iterCount++;
9                 if (distMatrix[i][k] == INF
10                    || distMatrix[k][j] == INF) continue;
11                 T newDist = distMatrix[i][k] + distMatrix[k][j];
12                 if (newDist < distMatrix[i][j]) {
13                     distMatrix[i][j] = newDist;
14                     nextMatrix[i][j] = nextMatrix[i][k];
15                 }
16             }
17 }

```

Listing 9: Основной цикл Флойда-Уоршалла (warshall.cpp)

Путь восстанавливается через `getPath(from, to)`: начиная с `from`, следуем по `nextMatrix` до `to`. Алгоритмическая сложность: $O(n^3)$, пространственная: $O(n^2)$.

2.4.3. Сравнение алгоритмов по числу итераций

Пункт меню 11 запускает оба алгоритма — Фалкерсона и Флойда-Уоршалла — на одном и том же графе и сравнивает счётчики `iterCount`, которые инкрементируются при каждом просмотре ячейки матрицы. Это позволяет объективно оценить трудоёмкость без зависимости от аппаратного обеспечения.

2.5. Модуль `lab3`

2.5.1. `MaxFlow<T>`: алгоритм Форда–Фалкерсона (Эдмондс–Карп)

Вход: `Graph<T> const&` (конструктор), `int s, int t`.

Выход: `T` — значение максимального потока.

Описание: Конструктор инициализирует остаточный граф `cap` из весовой матрицы входного графа; отрицательные веса берутся по модулю, поскольку пропускные способности должны быть неотрицательными.

Вспомогательный метод `bfs(s, t)` ищет кратчайший увеличивающий путь в остаточной сети; ребро (v, u) учитывается только при $cap[v][u] > 0$. Возвращается путь от s до t (или пустой вектор, если пути нет).

Основной цикл `compute(s, t)` итеративно:

1. находит увеличивающий путь через BFS;
2. вычисляет узкое место — минимальную пропускную способность на пути;
3. обновляет остаточный граф: уменьшает прямые рёбра, увеличивает обратные;
4. прибавляет найденный поток к суммарному.

```
1 template <typename T>
2 T MaxFlow<T>::compute(int s, int t) {
3     T maxFlow = T{};
4     while (true) {
5         vector<int> path = bfs(s, t);
6         if (path.empty()) break;
7
8         T push = numeric_limits<T>::max();
9         for (int i = 0; i + 1 < (int)path.size(); i++)
10             push = min(push, cap[path[i]][path[i+1]]);
11
12         for (int i = 0; i + 1 < (int)path.size(); i++) {
13             cap[path[i]][path[i+1]] -= push;
14             cap[path[i+1]][path[i]] += push;
15         }
16         maxFlow += push;
17     }
18     return maxFlow;
19 }
```

Listing 10: Алгоритм Эдмондса–Карпа (maxflow.cpp)

Сложность: $O(V \cdot E^2)$, пространственная: $O(V^2)$.

2.5.2. MinCostFlow<T>: поток минимальной стоимости

Вход: Graph<T> const&, vector<vector<T>> costMatrix, int s, int t, T targetFlow.

Выход: пара (достигнутый поток, минимальная стоимость).

Описание: Конструктор инициализирует остаточные пропускные способности cap и матрицу стоимостей cost; для обратных рёбер стоимость берётся с обратным знаком.

На каждой итерации вспомогательный метод findMinCostPath(s, t):

1. строит вспомогательный граф residual из рёбер с ненулевой остаточной пропускной способностью, присваивая им соответствующие стоимости;
2. запускает FloydWarshall<T> на этом графе для поиска пути минимальной стоимости от s до t;
3. возвращает путь и его стоимость.

```
1 template <typename T>
2 pair<vector<int>, T> MinCostFlow<T>::findMinCostPath(
3     int s, int t) const
4 {
5     Graph<T> residual(n);
6     for (int i = 0; i < n; i++)
7         for (int j = 0; j < n; j++)
8             if (cap[i][j] > T{})
9                 residual.addEdge(i, j, cost[i][j]);
10
11     FloydWarshall<T> fw(residual);
12     fw.compute();
13     const auto& dist = fw.getDistMatrix();
14     const T INF_VAL = numeric_limits<T>::max() / 2;
15     if (dist[s][t] >= INF_VAL) return {{}, T{}};
16     return {fw.getPath(s, t), dist[s][t]};
17 }
```

Listing 11: Поиск пути минимальной стоимости (mincost.cpp)

По найденному пути определяется узкое место, поток направляется по пути, остаточный граф обновляется. Итерации продолжаются до достижения targetFlow или невозможности найти путь.

По условию задания целевой поток берётся как $\lfloor 2/3 \cdot F_{\max} \rfloor$. Сложность одной итерации: $O(n^3)$ (Фloyd-Уоршалл).

2.5.3. Генерация матриц пропускных способностей и стоимости

При генерации нового графа метод `Lab3UI::onNewGraph` создаёт матрицу пропускных способностей из весовой матрицы (по модулю) и генерирует случайную матрицу стоимостей для существующих рёбер с помощью дополнительного экземпляра `Generator<int>`.

2.6. Модуль lab4

2.6.1. Kirchhoff: число остовных деревьев

Вход: `Graph<int> const&` — неориентированный граф.

Выход: `long long` — число различных остовных деревьев.

Описание: Метод `count` строит матрицу Лапласа B размера $n \times n$: $B[i][j] = -1$ для смежных вершин $i \neq j$, $B[i][i]$ — степень вершины i . Смежность определяется по наличию ребра хотя бы в одном направлении ($\text{adj}[i][j] \neq 0$ или $\text{adj}[j][i] \neq 0$), что корректно работает для неориентированных графов.

Затем выделяется минор L_{00} размера $(n - 1) \times (n - 1)$ удалением нулевой строки и нулевого столбца. Его определитель вычисляется *алгоритмом Барейсса* — целочисленным аналогом метода Гаусса, позволяющим избежать ошибок округления при работе с целыми числами:

```
1 for (int k = 0; k < m; k++) {
2     // поиск ненулевого опорного элемента
3     int pivot = -1;
4     for (int i = k; i < m; i++)
5         if (mat[i][k] != 0) { pivot = i; break; }
6     if (pivot == -1) return 0;
7     if (pivot != k) swap(mat[k], mat[pivot]);
8
9     for (int i = k + 1; i < m; i++) {
10        for (int j = k + 1; j < m; j++) {
11            mat[i][j] = mat[k][k] * mat[i][j]
12                - mat[i][k] * mat[k][j];
13            if (k > 0)
14                mat[i][j] /= mat[k-1][k-1];
15        }
16        mat[i][k] = 0;
17    }
18 }
19 return abs(mat[m-1][m-1]);
```

Listing 12: Алгоритм Барейсса для вычисления определителя (kirchhoff.cpp)

Алгоритм Барейсса на каждом шаге k делит элементы на $\text{mat}[k-1][k-1]$, что гарантирует целочисленность промежуточных результатов. Сложность: $O(n^3)$, пространственная: $O(n^2)$.

2.6.2. Kruskal: алгоритм Краскала

Вход: `Graph<int> const&` — связный неориентированный взвешенный граф.

Выход: `vector<Edge>` — рёбра минимального остова; пустой вектор если граф несвязен.

Описание: Конструктор собирает все рёбра (каждое ровно один раз, условие $i < j$) в вектор `edges` с весами по модулю.

Метод `compute()` сортирует рёбра по весу и поочерёдно добавляет их в остова, если они не создают цикла. Проверка цикличности выполняется через BFS по уже построенному остову: если вершины u и v уже связаны, ребро пропускается.

```
1 vector<Edge> Kruskal::compute() {
2     sort(edges.begin(), edges.end());
3     vector<Edge> T;
4     Graph<int> tree(n);
5     for (auto& e : edges) {
6         if ((int)T.size() == n - 1) break;
7         if (hasPath(e.u, e.v, tree)) continue;
8         T.push_back(e);
9         tree.addEdge(e.u, e.v, e.w);
10        tree.addEdge(e.v, e.u, e.w);
11    }
12    return ((int)T.size() == n - 1) ? T : vector<Edge>{};
13 }
```

Listing 13: Основной цикл алгоритма Краскала (kruskal.cpp)

Метод `hasPath` реализует BFS по матрице смежности текущего остова. Сложность: $O(E^2)$ в худшем случае (BFS на каждое ребро), пространственная: $O(V + E)$.

2.6.3. Prufer: код Прюфера

Вход (encode): `vector<Edge> const&` — рёбра МОД, `int n`.

Выход (encode): `PruferCode` — структура с полями: `seq` (последовательность длины $n - 2$), `weights` (веса рёбер), `n` (число вершин).

Описание (encode): Строится временный граф `tree` из рёбер МОД. Ведётся вектор степеней `d` и вектор активности вершин `V`. На каждом из $n - 1$ шагов линейным поиском находится активная висющаяся вершина с минимальным номером ($d[v] = 1$); её единственный сосед записывается в `seq`, вес соответствующего ребра — в `weights`. Оба конца уменьшают степень, вершина v деактивируется.

```
1 PruferCode Prufer::encode(const vector<Edge>& mst, int p) {
2     // строим дерево, считаем степени
3     for (int i = 0; i < p - 1; i++) {
4         int v = -1;
5         for (int k = 0; k < p; k++)
```

```

6         if (V[k] && d[k] == 1) { v = k; break; }
7         int neighbor = -1, weight = 0;
8         for (int to = 0; to < p; to++)
9             if (V[to] && adj[v][to] != 0)
10                { neighbor = to; weight = wgt[v][to]; break; }
11         code.seq.push_back(neighbor);
12         code.weights.push_back(weight);
13         V[v] = false; d[v]--; d[neighbor]--;
14     }
15     return code;
16 }

```

Listing 14: Кодирование в код Прюфера с сохранением весов (prufer.cpp)

Описание (decode): Для каждого элемента $seq[i]$ линейным поиском находится наименьшая активная вершина, не встречающаяся в остатке кода $seq[i..n-2]$. Между ней и $seq[i]$ добавляется ребро с весом $weights[i]$. После $n - 2$ шагов оставшиеся две активные вершины соединяются последним ребром. Сложность: $O(n^2)$, пространственная: $O(n)$.

2.6.4. MIS: максимальное независимое множество

Вход: `vector<vector<int>> const&` — матрица смежности (исходного графа или МОД — по выбору пользователя).

Выход: `vector<int>` — вершины МНМ.

Описание: Метод `compute()` запускает рекурсивный `backtrack(S, T)`, где S — текущее независимое множество, T — кандидаты. Для каждой вершины $v \in T$ проверяется независимость: если v не смежна ни с одной вершиной из S , она добавляется и формируется новый список кандидатов `newT` без соседей v . Если найденное множество $S \cup \{v\}$ больше текущего максимума, оно сохраняется.

```

1 void MIS::backtrack(vector<int> S, vector<int> T) {
2     for (int v : T) {
3         if (isIndependent(S, v)) {
4             vector<int> S_plus_v = S;
5             S_plus_v.push_back(v);
6             if ((int)S_plus_v.size() > m) {
7                 X = S_plus_v;
8                 m = S_plus_v.size();
9             }
10            vector<int> newT;
11            for (int u : T)
12                if (u != v && adj[v][u] == 0 && adj[u][v] == 0)
13                    newT.push_back(u);
14            backtrack(S_plus_v, newT);
15        }
16    }
17 }

```

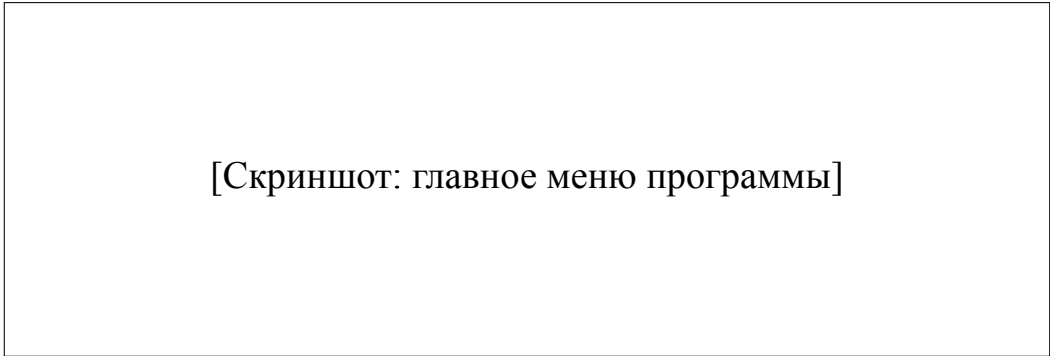
Listing 15: Backtracking для нахождения MIS (mis.cpp)

Метод `isIndependent (S, v)` проверяет, что ни одна вершина из S не смежна с v в любом направлении. Сложность: $O(2^n)$ в худшем случае, пространственная: $O(n)$.

3. Результаты работы программы

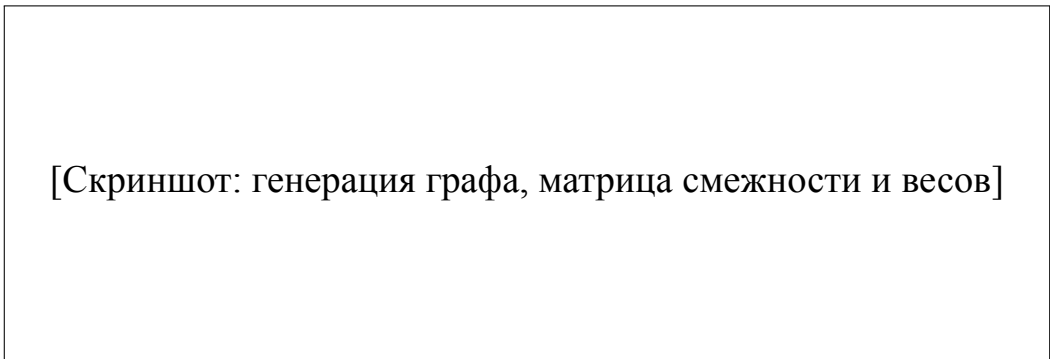
3.1. Общее меню и генерация графа

При запуске программы пользователю предлагается консольное меню, разбитое на четыре раздела по лабораторным работам. Для начала работы необходимо выбрать пункт 1, ввести количество вершин, тип ориентированности и тип весов рёбер.



[Скриншот: главное меню программы]

Рис. 2: Главное меню программы



[Скриншот: генерация графа, матрица смежности и весов]

Рис. 3: Генерация графа, матрица смежности и матрица весов

3.2. Лабораторная работа 1

3.3. Лабораторная работа 2

3.4. Лабораторная работа 3

3.5. Лабораторная работа 4

[Скриншот: эксцентриситеты, центр, диаметр]

Рис. 4: Эксцентриситеты вершин, центр и диаметр

[Скриншот: матрица Шимбелла (мин/макс пути)]

Рис. 5: Матрица Шимбелла: кратчайшие и наидлиннейшие пути длиной k

[Скриншот: все пути от A до B]

Рис. 6: Все простые маршруты от вершины A до вершины B

[Скриншот: алгоритм Фалкерсона, группы вершин, новая матрица]

Рис. 7: Топологическая сортировка алгоритмом Фалкерсона

[Скриншот: матрица расстояний, кратчайший путь]

Рис. 8: Алгоритм Флойда-Уоршалла: матрица расстояний и кратчайший путь

[Скриншот: сравнение числа итераций Фалкерсона и Флойда]

Рис. 9: Сравнение алгоритмов по числу итераций

[Скриншот: пошаговое нахождение максимального потока]

Рис. 10: Нахождение максимального потока алгоритмом Форда–Фалкерсона

[Скриншот: поток минимальной стоимости, итерации]

Рис. 11: Поток минимальной стоимости ($\lfloor 2/3 \cdot F_{\max} \rfloor$)

[Скриншот: матрица Лапласа, число остовных деревьев]

Рис. 12: Матрица Лапласа и число остовных деревьев (теорема Кирхгофа)

[Скриншот: рёбра МОД и суммарный вес]

Рис. 13: Минимальный остов алгоритмом Краскала

[Скриншот: код Прюфера и декодированный остов]

Рис. 14: Кодирование и декодирование кода Прюфера

[Скриншот: максимальное независимое множество]

Рис. 15: Максимальное независимое множество вершин

Заключение

В ходе выполнения четырёх лабораторных работ были реализованы все поставленные задания. В лабораторной работе 1 выполнена случайная генерация связного ациклического ориентированного графа по распределению Вейбулла, вычислены эксцентриситеты вершин, выделены центр и диаметральные вершины, реализован метод Шимбелла для поиска кратчайших и наидлиннейших маршрутов заданной длины, а также определение всех простых путей между двумя вершинами. В лабораторной работе 2 реализованы топологическая сортировка по алгоритму Фалкерсона, нахождение кратчайших путей алгоритмом Флойда-Уоршалла с выводом матрицы расстояний и сравнение алгоритмов по числу итераций. В лабораторной работе 3 построена транспортная сеть, найден максимальный поток алгоритмом Форда–Фалкерсона и вычислен поток минимальной стоимости величиной $\lfloor 2/3 \cdot F_{\max} \rfloor$. В лабораторной работе 4 подсчитано число остовных деревьев по теореме Кирхгофа, построен минимальный остов алгоритмом Краскала, выполнено кодирование и декодирование кода Прюфера с сохранением весов рёбер, а также найдено максимальное независимое множество вершин.

Достоинства реализации

Все четыре лабораторные работы используют один и тот же сгенерированный граф без регенерации, что удобно при тестировании разных алгоритмов на одних и тех же данных.

Алгоритм поиска кратчайшего пути по стоимости в третьей лабораторной переиспользует тот же алгоритм Флойда-Уоршалла, что реализован во второй. Это позволило не писать один и тот же код дважды.

Генерация весов рёбер и перемешивание вершин при построении графа используют одно и то же распределение Вейбулла, что делает результат более случайным и непредсказуемым по сравнению с равномерным генератором.

Недостатки реализации

Граф хранится в виде матриц, поэтому при большом числе вершин программа занимает много памяти, даже если рёбер мало.

В алгоритме Краскала проверка на цикл выполняется обходом уже построенного остова, что работает медленнее, чем специальные структуры данных для этой задачи.

Поиск всех путей и максимального независимого множества работает очень долго на больших графах, так как перебирает все возможные варианты. Никакого ограничения на размер входных данных не предусмотрено.

Масштабирование (что можно добавить в дальнейшем)

Можно заменить матричное представление графа на списки смежности, чтобы программа работала быстрее и занимала меньше памяти на разреженных графах.

Добавление графического интерфейса позволило бы отображать граф наглядно: рисовать вершины и рёбра, выделять цветом центр и диаметральные вершины, показывать пошагово работу алгоритмов и менять параметры без перезапуска программы.

Список литературы

- [1] Секция «Телематика» [Электронный ресурс]. — URL: <https://tema.spbstu.ru/dismath/> (дата обращения: 16.05.2026).
- [2] Новиков Ф. А. *Дискретная математика для программистов*. — 3-е изд. — СПб.: Питер Пресс, 2009. — 384 с.
- [3] Вадзинский Р. Н. *Справочник по вероятностным распределениям*. — СПб.: Наука, 2001. — 295 с.
- [4] Алгоритм Краскала [Электронный ресурс] // Викиконспекты ИТМО. — URL: https://neerc.ifmo.ru/wiki/index.php?title=%D0%90%D0%BB%D0%B3%D0%BE%D1%80%D0%B8%D1%82%D0%BC_%D0%9A%D1%80%D0%B0%D1%81%D0%BA%D0%B0%D0%BB%D0%B0 (дата обращения: 16.05.2026).